



Information Security

java-cryptography Xor_Opposite algorithm

Table of Contents:

1	INTRODUCTION
2	ENCRYPTION
3	DECRYPTION
4	KEY
5	Hash Generation and Integrity Verification
6	BLOCK DIAGRAM
7	FLOW CHARTS
8	SCREENSHOTS OF THE OUTPUTS
9	REFERENCES
10	APPENDIX

Introduction:

In this project we aim for a transparent and highly efficient algorithm based on the integration of two key steps: XOR and microflow architecture. This approach focuses on enhancing data protection by incorporating ambiguity and complexity within the ciphertext. XOR provides the advantage of unpredictability while making it more difficult for various parties to engage in unauthorized decryption. By combining these two processes, this design supports robust security systems, reduces the likelihood of data breaches, and offers an easy-to-implement method for controlling data of your choice.

ENCRYPTION:

1. Convert the plaintext and the key into bytes (binary form).
2. Determine the rotation shift value from the first character of the key using:
 $\text{shift} = \text{key}[0] - 65$.
3. Rotate each byte of the plaintext to the left using the calculated shift value.
4. Ensure the key matches the length of the plaintext by repeating it if necessary.
5. XOR the rotated plaintext bytes with the corresponding key bytes.
6. Encode the resulting ciphertext bytes using Base64.
7. Compute the SHA-256 hash of the plaintext bytes and represent it in hexadecimal.
8. Compute the SHA-256 hash of the ciphertext bytes and represent it in hexadecimal.
9. Return the Base64 ciphertext along with both SHA-256 hashes. [1][2].

DECRYPTION:

1. **Byte Conversion:** Transform both the Base64-decoded encrypted text and the key into byte sequences.
2. **Shift Calculation:** Compute the rotation value from the key's initial character using:
 $\text{shift} = \text{key}[0] - 65$.
3. **XOR Process:** Apply XOR between every encrypted byte and its matching byte from the key.
4. **Right Rotation:** Rotate each resulting byte to the right according to the calculated shift amount.
5. **Plaintext Reconstruction:** Change the processed bytes back into readable plaintext symbols.
6. **Final Result:** Output the recovered plain message.[3][4].

KEY:

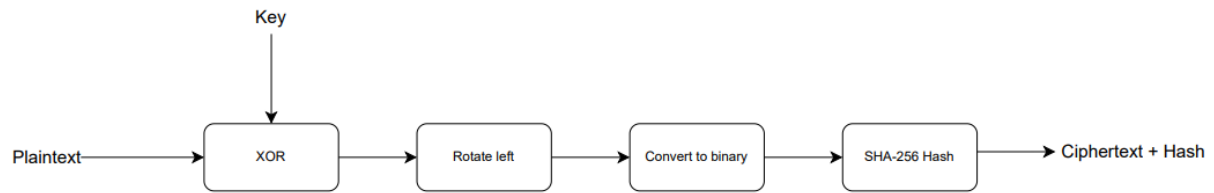
The key is converted to bytes and used in XOR and rotation .

Hash Generation and Integrity Verification:

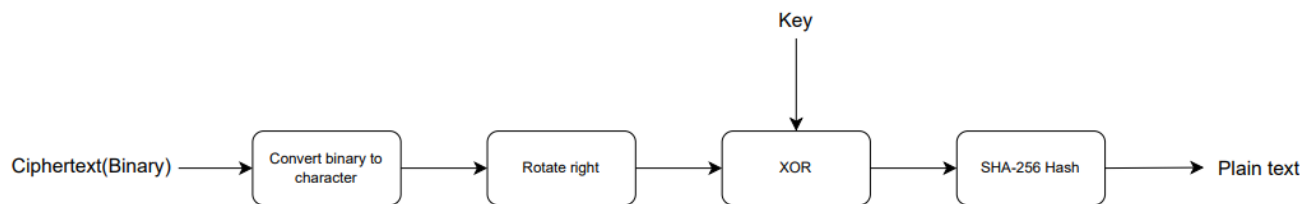
In this stage, the system creates a SHA-256 hash from the ciphertext after encryption. The hash acts like a fingerprint that represents the encrypted data.

To check integrity, the program generates the SHA-256 hash of the ciphertext again and compares it with the original hash. If both hashes are the same, it means the ciphertext was not changed. If they are different, it means the data was modified or corrupted.[5]

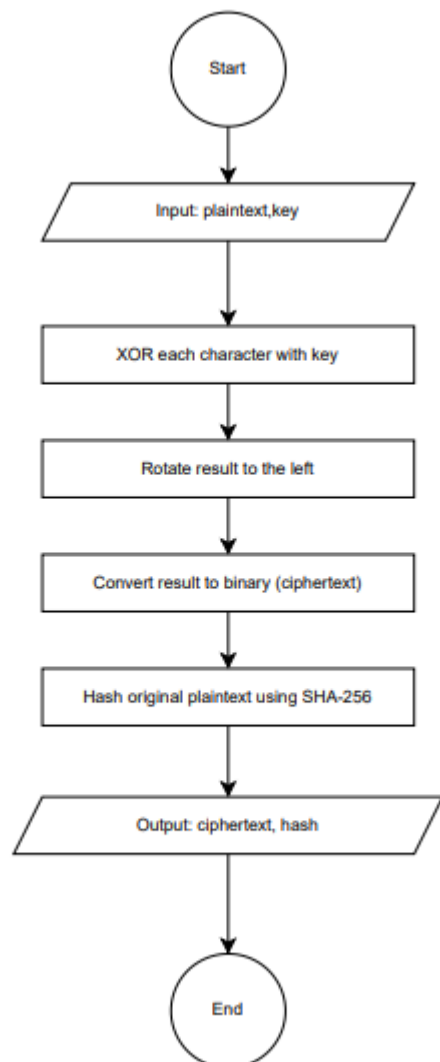
BLOCK DIAGRAM ENCRYPTION:



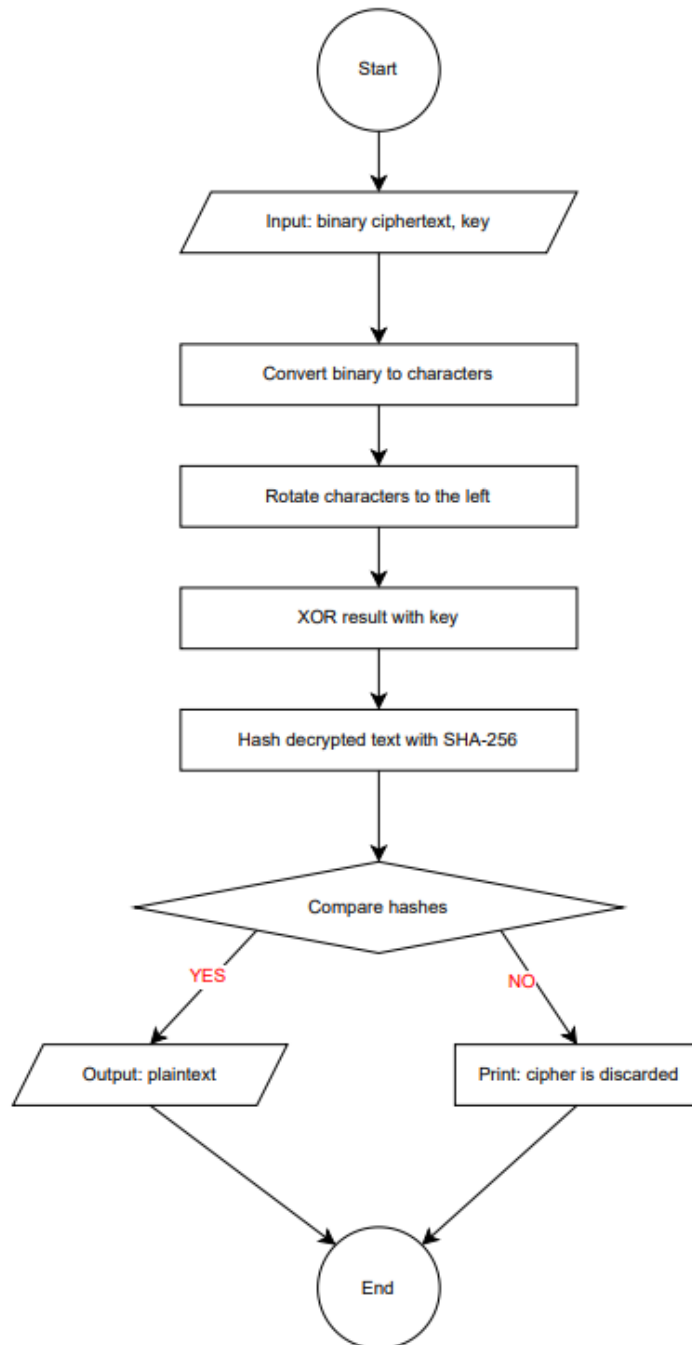
BLOCK DIAGRAM DECRYPTION:



ENCRYPTION:



DECRYPTION:



SCREENSHOTS OF THE OUTPUTS ENCRYPTION:

```
Output - InfoSecurityProject (run)

run:
-----
1 - Encrypt Text
2 - Exit
-----
Select an option: 1
Enter the plaintext: renad
Enter key: ttttt

Ciphertext (Base64): xn4mXnY=
SHA-256 of plaintext: 3d2ba7e838ef08050cb4723e3139d033ba8408b94b63272354d2b44320520308
SHA-256 of ciphertext: 0c73692901e8a98f74ceab2b2f725c1efe33d4df283cc27567a3a91363683324

-----
-----
1 - Encrypt Text
2 - Exit
-----
Select an option: 2
Program will now exit.
BUILD SUCCESSFUL (total time: 9 minutes 3 seconds)
```

Hash Generation and Integrity Verification:

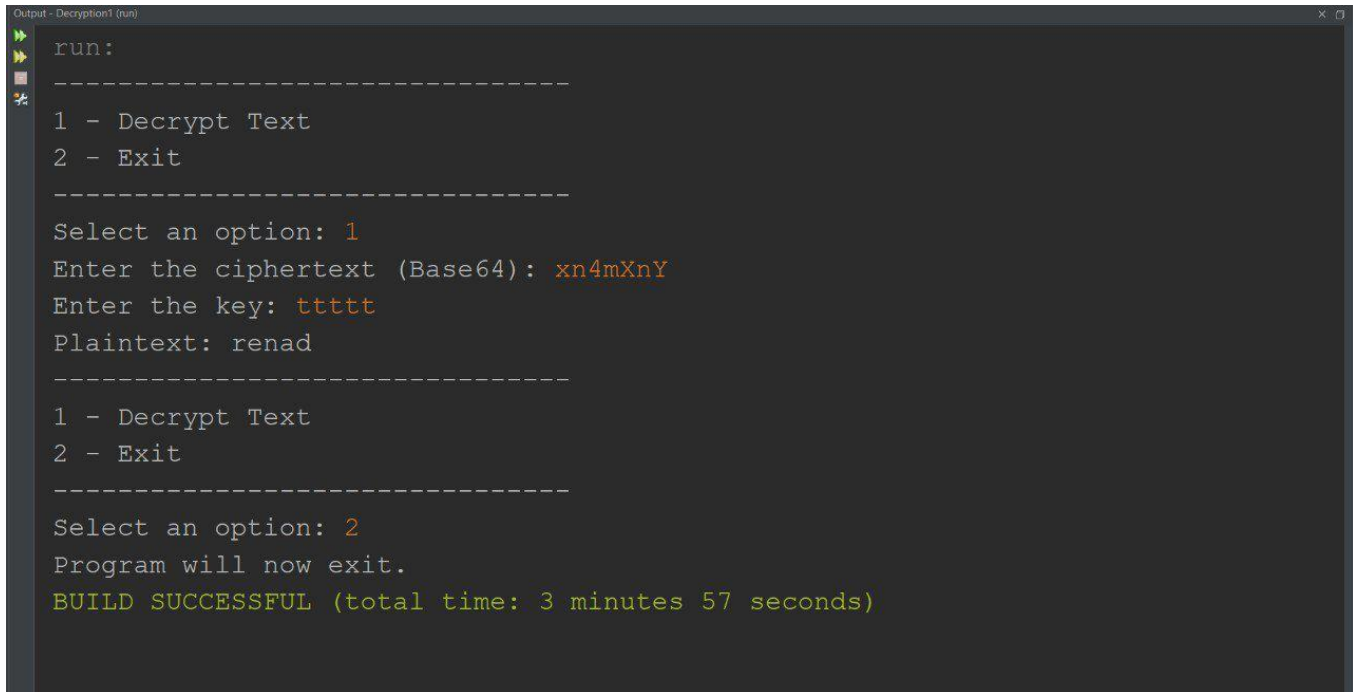
```
Output - project1 (run) #2 x

run:
-----
1 - Encrypt Text
2 - Exit
-----
Select an option: 1
Enter the plaintext: Lara
Enter key: TTTT

Ciphertext (Base64): Nl7GXg==
SHA-256 Hash of Ciphertext: 910b60539909db57aca8755cc0802a4807167c06b3b46725e3990483d8c3ef9c
Ciphertext Integrity Verified? true

-----
-----
1 - Encrypt Text
2 - Exit
-----
Select an option: 2
Program will now exit.
BUILD SUCCESSFUL (total time: 6 minutes 33 seconds)
```

SCREENSHOTS OF THE OUTPUTS DECRYPTION:



```
Output - Decryption1 (run)
run:
-----
1 - Decrypt Text
2 - Exit
-----
Select an option: 1
Enter the ciphertext (Base64): xn4mXnY
Enter the key: ttttt
Plaintext: renad
-----
1 - Decrypt Text
2 - Exit
-----
Select an option: 2
Program will now exit.
BUILD SUCCESSFUL (total time: 3 minutes 57 seconds)
```


Reference :

1. <https://www.geeksforgeeks.org/bitwise-xor-operator-in-programming/>
2. <https://www.geeksforgeeks.org/dsa/rotate-bits-of-an-integer/>
3. <https://stackoverflow.com/questions/66283799/java-xor-decryption>
4. <https://www.geeksforgeeks.org/dsa/xor-cipher/>
5. Srinam. (2025, July 23). Bitwise XOR Operator in Programming. GeeksforGeeks.
<https://www.geeksforgeeks.org/bitwise-xor-operator-in-programming/>

APPENDIX:

```
        ;package infosecurityproject
;import java.nio.charset.StandardCharsets
        ;import java.security.MessageDigest
        ;import java.util.Base64
        ;import java.util.Scanner

        } public class InfoSecurityProject

        } public static int getShiftKey(String key)
        ;int shiftKey = key.charAt(0) - 65
        ;return shiftKey
        {

        } private static byte rotateLeft(byte b, int shift)
        ;int s = shift & 0x07
        ;int v = (b & 0xFF)
        ;int res = ((v << s) | (v >>> (8 - s))) & 0xFF
        ;return (byte) res
        {

        ENCRYPTION //

} public static byte[] encrypt(byte[] data, byte[] key, int rotateShift)
        ;byte[] out = new byte[data.length]
        } for (int i = 0; i < data.length; i++)
        ;byte d = data[i]
        byte r = rotateLeft(d, rotateShift); //first algorithm=Rotation left
        ;byte k = key[i % key.length]
        out[i] = (byte) (r ^ k); //Second algorithm=XOR
        {
        ;return out
        {

        } public static String sha256Hex(byte[] data)
        } try
;MessageDigest md = MessageDigest.getInstance("SHA-256")
        ;byte[] hash = md.digest(data)
        ;()StringBuilder sb = new StringBuilder
        } for (byte b : hash)
        ;sb.append(String.format("%02x", b))
        {
        ;()return sb.toString
        } (catch (Exception e {
;throw new RuntimeException("SHA-256 Error", e)
        {
        {

        } ()private static void printOptions
; ("-----")System.out.println
```

```

        ;System.out.println("1 - Encrypt Text")
        ;System.out.println("2 - Exit")
; ("-----")System.out.println
        ;System.out.print("Select an option: ")
        {

} public static void main(String[] args)
;Scanner input = new Scanner(System.in)
;String userChoice

        } while (true)
        ;()printOptions
        ;()userChoice = input.nextLine().trim

        } if (userChoice.equals("2"))
;System.out.println("Program will now exit.")
        ;break
        {

        } switch (userChoice)
        { : "case "1
        ;System.out.print("Enter the plaintext: ")
;()String plaintext = input.nextLine().toUpperCase

        ;System.out.print("Enter key: ")
;()String keyStr = input.nextLine().toUpperCase

        ;int shiftKey = getShiftKey(keyStr)

;byte[] plainBytes = plaintext.getBytes(StandardCharsets.UTF_8)
;byte[] keyBytes = keyStr.getBytes(StandardCharsets.UTF_8)

;byte[] cipherBytes = encrypt(plainBytes, keyBytes, shiftKey)
;String cipherB64 = Base64.getEncoder().encodeToString(cipherBytes)

        ;System.out.println("\nCiphertext (Base64): " + cipherB64)
;System.out.println("SHA-256 of plaintext: " + sha256Hex(plainBytes))
;System.out.println("SHA-256 of ciphertext: " + sha256Hex(cipherBytes))

        ;System.out.println("\n-----")
        ;break
        {
        :default
;System.out.println("Invalid option! Please try again.\n")
        ;break
        {
        {
        ;()input.close
        {
        {

```

```

        ;package InfoSecurityProject01
;import java.nio.charset.StandardCharsets
;import java.security.MessageDigest
;import java.util.Base64
;import java.util.Scanner

    } public class InfoSecurityProject

    } public static int getShiftKey(String key)
        ;int shiftKey = key.charAt(0) - 65
        ;return shiftKey
        {

    } private static byte rotateLeft(byte b, int shift)
        ;int s = shift & 0x07
        ;int v = (b & 0xFF)
;int res = ((v << s) | (v >>> (8 - s))) & 0xFF
        ;return (byte) res
        {

            ENCRYPTION //
} public static byte[] encrypt(byte[] data, byte[] key, int rotateShift)
        ;byte[] out = new byte[data.length]
        } for (int i = 0; i < data.length; i++)
            ;byte d = data[i]
byte r = rotateLeft(d, rotateShift); //first algorithm=Rotation left
            ;byte k = key[i % key.length]
            out[i] = (byte) (r ^ k); //Second algorithm=XOR
            {
                ;return out
            {

                SHA-256 //
    } public static String sha256Hex(byte[] data)
        } try
;MessageDigest md = MessageDigest.getInstance("SHA-256")
        ;byte[] hash = md.digest(data)
        ;()StringBuilder sb = new StringBuilder
        } for (byte b : hash)
            ;sb.append(String.format("%02x", b))
            {
                ;()return sb.toString
            } (catch (Exception e {
;throw new RuntimeException("SHA-256 Error", e)
            {
                {

----- HASHING THE CIPHERTEXT ----- //
    } public static String generateCipherHash(byte[] cipherBytes)
        ;return sha256Hex(cipherBytes)
        {

----- VERIFICATION ----- //

```

```

} public static boolean verifyCipherHash(String originalHash, byte[] cipherBytes)
    ;String newHash = generateCipherHash(cipherBytes)
    ;return newHash.equals(originalHash)
    {

        } ()private static void printOptions
; ("-----")System.out.println
    ;System.out.println("1 - Encrypt Text")
    ;System.out.println("2 - Exit")
; ("-----")System.out.println
    ;System.out.print("Select an option: ")
    {

        } public static void main(String[] args)
        ;Scanner input = new Scanner(System.in)
        ;String userChoice

            } while (true)
            ; ()printOptions
            ; ()userChoice = input.nextLine().trim

                } if (userChoice.equals("2"))
                ;System.out.println("Program will now exit.")
                ;break
                {

                    } switch (userChoice)
                    } : "case "1
                ;System.out.print("Enter the plaintext: ")
                ; ()String plaintext = input.nextLine().toUpperCase

                    ;System.out.print("Enter key: ")
                    ; ()String keyStr = input.nextLine().toUpperCase

                        ;int shiftKey = getShiftKey(keyStr)

;byte[] plainBytes = plaintext.getBytes(StandardCharsets.UTF_8)
;byte[] keyBytes = keyStr.getBytes(StandardCharsets.UTF_8)

                ENCRYPT //
                ;byte[] cipherBytes = encrypt(plainBytes, keyBytes, shiftKey)
;String cipherB64 = Base64.getEncoder().encodeToString(cipherBytes)

                ;System.out.println("\nCiphertext (Base64): " + cipherB64)

                ----- HASH ON CIPHERTEXT ----- //
                ;String cipherHash = generateCipherHash(cipherBytes)
                ;System.out.println("SHA-256 Hash of Ciphertext: " + cipherHash)

                ----- VERIFY INTEGRITY ----- //
                ;boolean verified = verifyCipherHash(cipherHash, cipherBytes)
                ;System.out.println("Ciphertext Integrity Verified? " + verified)

```

```

;System.out.println("\n-----")
        ;break
        {

        :default
;System.out.println("Invalid option! Please try again.\n")
        ;break
        {
        {
        ; ()input.close
        {
        {

        */
Click nbfs://nbhost/SystemFileSystem/Templates/Licenses/license-default.txt to  *
change this license
Click nbfs://nbhost/SystemFileSystem/Templates/Classes/Main.java to edit this  *
template
/*
;package Decryption1

        **/
        *
        author dinaa@ *
        /*

;import java.util.Base64
;import java.util.Scanner

} public class Decryption1

} private static byte rotateRight(byte b, int shift)
        ;int s = shift & 0x07
        ;int v = (b & 0xFF)
;int res = ((v >>> s) | (v << (8 - s))) & 0xFF
        ;return (byte) res
        {

} public static byte[] decrypt(byte[] cipherData, byte[] key, int rotateShift)
        ;byte[] out = new byte[cipherData.length]
        } for (int i = 0; i < cipherData.length; i++)
                ;byte c = cipherData[i]
                ;byte k = key[i % key.length]
                ;byte r = (byte) (c ^ k)
                ;out[i] = rotateRight(r, rotateShift)
                {
                ;return out
                {

        } public static void main(String[] args)
;Scanner scanner = new Scanner(System.in)

```

```

        } while (true)
; ("-----")System.out.println
    ;System.out.println("1 - Decrypt Text")
    ;System.out.println("2 - Exit")
; ("-----")System.out.println
    ;System.out.print("Select an option: ")
    ;()int choice = scanner.nextInt
    ;()scanner.nextLine

        } if (choice == 1)
;System.out.print("Enter the ciphertext (Base64): ")
    ;()String cipherText = scanner.nextLine
    ;System.out.print("Enter the key: ")
    ;()String key = scanner.nextLine().toUpperCase
    ;int rotationValue = key.charAt(0) - 65
;byte[] cipherBytes = Base64.getDecoder().decode(cipherText)
    ;()byte[] keyBytes = key.getBytes
;byte[] plainBytes = decrypt(cipherBytes, keyBytes, rotationValue)
    ;String plainText = new String(plainBytes)
;System.out.println("Plaintext: " + plainText.toLowerCase())
    } (else if (choice == 2 {
    ;System.out.println("Program will now exit.")
        ;break
    } else {
;System.out.println("Invalid option! Try again.\n")
        {
        {
            ;()scanner.close
            {
            {

```